

Utilizando el SGBD adecuado:

- Crea una base de datos que se llame Ferretería
- Crea una tabla se llame artículos con 4 campos que todos sean con inserción obligatoria
 - **idArticulo**: clave primaria (no es autincremental)
 - **descripcion**: dato de tipo cadena
 - **pvp**: dato de tipo decimal
 - **stock**: dato de tipo entero
- Inserta un procedimiento almacenado que inserte un artículo pasándole los parámetros adecuados. Ejemplo de llamada: **call insertarArticulo(4,'Martillo percutor', 1900.4,2)**
- Crear un usuario y contraseña segura con **GRANT ALL PRIVILEGES**

Realiza un programa en Java que haga lo siguiente:

- Prepara el proyecto para la gestión de la base de datos de Ferretería
- Realiza la conexión con el usuario y contraseña adecuada.
- Realiza modificaciones en los registros sin interacción con el usuario
 - Borra todos los registros
 - Inserta 3 registros directamente, utiliza el ignore en la sentencia INSERT para evitar problemas al principio
 - Muestra todos los registros de la tabla
 - Modifica un registro buscando por id
 - Muestra todos los registros de la tabla
- Crea un método de mostrar todos los registros (Muestra todos los registros de la tabla). Tienes que pasar por parámetro la conexión
- Realiza modificaciones en los registros interactuando con el usuario:
 - Pide datos al usuario e inserta un registros
 - Muestra todos los registros de la tabla
 - Pide datos al usuario y muestra por ID
 - Pide datos al usuario y borra un registro por ID
 - Muestra todos los registros de la tabla
- Llama al procedimiento almacenado con datos que se pide al usuario

Hay que utilizar en todo momento try-with-resources

Desarrollo del reto

Para la realización de este reto se ha utilizado **MySQL como sistema gestor de bases de datos**, gestionado mediante **phpMyAdmin**, junto con **XAMPP** para el arranque del servidor Apache y MySQL, que trabaja a través del puerto 3306.

El objetivo del ejercicio es poner en práctica la creación y gestión de una base de datos y su posterior acceso desde un programa Java utilizando JDBC.

Creación de la base de datos, la tabla artículos con sus campos y tipos de datos.

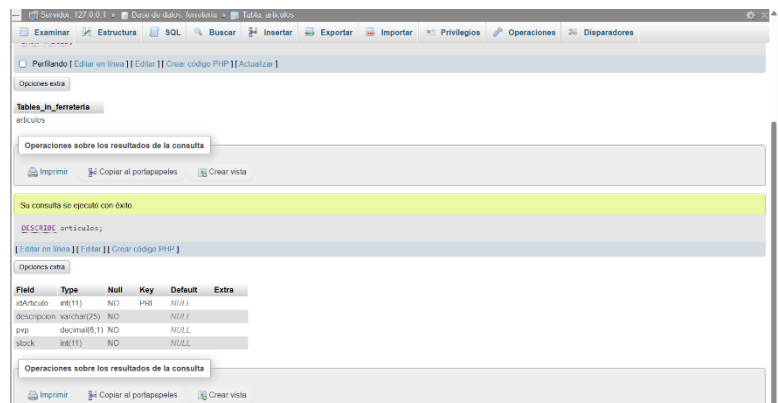
En primer lugar, se creó la **base de datos ferreteria**. A continuación, dentro de esta base de datos se definió la **tabla artículos**, formada por cuatro campos obligatorios, tal y como indicaba el enunciado del reto.

El campo idArticulo se definió como **clave primaria** y no autoincremental. El campo descripcion se configuró como un tipo de dato de texto, mientras que pvp y stock se definieron como valores numéricos. El campo pvp se estableció como DECIMAL(6,1) para poder trabajar con precios elevados sin que se produjeran errores durante las inserciones.

A continuación, se muestra el código SQL utilizado para la creación de la base de datos ferreteria y la tabla artículos.

```
CREATE DATABASE  
FERRETERIA;
```

```
CREATE TABLE ARTICULOS(  
idArticulo INT PRIMARY KEY  
NOT NULL,  
    descripcion VARCHAR(25)  
NOT NULL,  
    pvp DECIMAL(6,1) NOT  
NULL,  
    stock INT NOT NULL  
);
```



Creación del procedimiento almacenado

Una vez creada la tabla, se desarrolló un **procedimiento almacenado** llamado insertarArticulo. Este procedimiento permite insertar un nuevo artículo en la tabla articulos recibiendo los datos necesarios como parámetros.

El uso de un procedimiento almacenado facilita la inserción de datos desde Java sin necesidad de escribir directamente la sentencia INSERT.

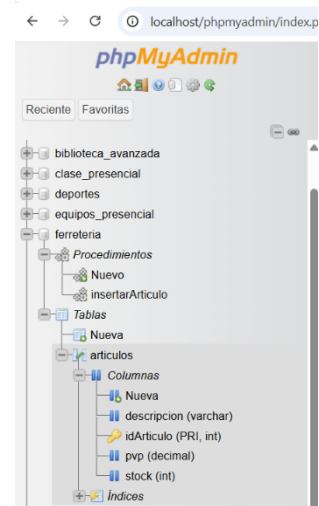
Para comprobar su correcto funcionamiento, se realizó una prueba ejecutando una llamada CALL insertarArticulo(...) desde phpMyAdmin, verificando que el registro se insertaba correctamente.

Procedimiento insertarArticulo:

```
DELIMITER //
```

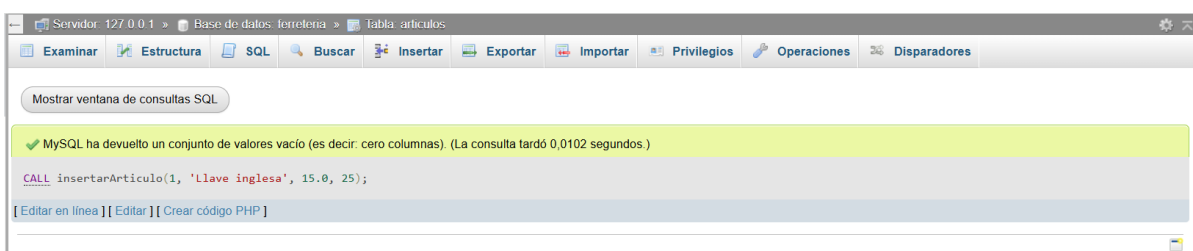
```
CREATE PROCEDURE insertarArticulo(  
  IN p_idArticulo INT,  
  IN p_descripcion VARCHAR(25),  
  IN p_pvp DECIMAL(6,1),  
  IN p_stock INT  
)  
BEGIN  
  INSERT INTO ARTICULOS (idArticulo, descripcion,  
pvp, stock)  
  VALUES (p_idArticulo, p_descripcion, p_pvp, p_stock);  
END //
```

```
DELIMITER ;
```



Ejecución de una llamada CALL insertarArticulo(...) desde phpMyAdmin.

CALL insertarArticulo(1, 'Llave inglesa', 15.0, 25);



Creación del usuario

También se creó un usuario específico con contraseña segura y permisos completos sobre la base de datos ferreteria mediante la sentencia GRANT ALL PRIVILEGES.

Debido a problemas de configuración del entorno, la conexión desde el programa Java se realizó finalmente con el usuario por defecto, dejando constancia de que se conoce el proceso correcto de creación de usuarios y asignación de permisos.

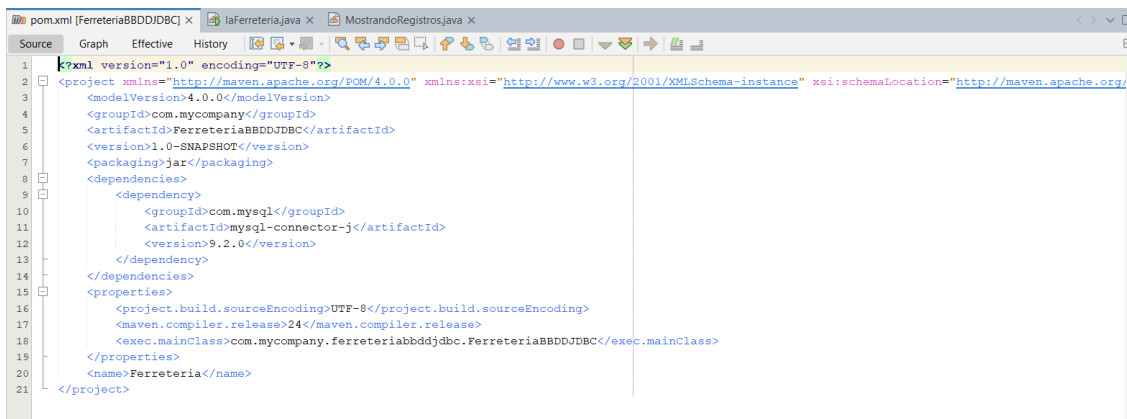
```
CREATE USER IF NOT EXISTS 'selma'@'localhost' IDENTIFIED BY 'selma26*';  
GRANT ALL PRIVILEGES ON ferreteria.* TO 'selma'@'localhost';  
FLUSH PRIVILEGES;
```

Preparación del proyecto Java

Para el desarrollo del programa se creó un **proyecto Java**, configurado para trabajar con MySQL mediante JDBC.

Se establecieron los parámetros de conexión a la base de datos y se utilizó **try-with-resources** para garantizar el cierre correcto de la conexión y del resto de recursos utilizados.

```
String url = "jdbc:mysql://localhost:3306/ferreteria";  
String user = "root";  
String password = "";
```



Operaciones sin interacción con el usuario

En la primera parte del programa se realizaron varias operaciones sobre la base de datos **sin intervención del usuario**, ya que se ejecutan automáticamente al iniciar la aplicación.

Estas operaciones incluyen el **borrado de todos los registros** de la tabla, la **inserción de tres artículos directamente** utilizando INSERT IGNORE, la **visualización de todos los registros** y la **modificación de un artículo buscando por su identificador**.

Tras cada una de estas acciones, se muestran los registros de la tabla para comprobar que los cambios se han realizado correctamente.

```
Output - Run (laFerreteria) x
WARNING: sun.misc.Unsafe:staticFieldBase will be removed in a future release
Scanning for projects...

-----< com.mycompany:FerreteriaBDDJDBC >-----
Building Ferreteria 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

--- resources:3.3.1:resources (default-resources) @ FerreteriaBDDJDBC ---
skip non existing resourceDirectory C:\Users\selma\Documents\NetBeansProjects\FerreteriaBDDJDBC\src\main\resources

--- compiler:3.13.0:compile (default-compile) @ FerreteriaBDDJDBC ---
Nothing to compile - all classes are up to date.

--- exec:3.1.0:exec (default-cli) @ FerreteriaBDDJDBC ---
Borrado hecho

Inserción de 3 registros hecha

TABLA ARTICULOS (mediante método)
-----
idArticulo: 2, descripción: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripción: Tornillos 5mm, pvp: 9.9, stock: 200
idArticulo: 4, descripción: Cinta aislante, pvp: 2.2, stock: 80

Registros modificados: 1

TABLA ARTICULOS (mediante método)
-----
idArticulo: 2, descripción: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripción: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripción: Cinta aislante, pvp: 2.2, stock: 80

Inserta un artículo (id):
```

Método para mostrar los registros

Para mostrar el contenido de la tabla se creó un método independiente al que se le pasa la conexión como parámetro.

Este método se reutiliza en distintas partes del programa, lo que permite mantener el código más ordenado y evitar la repetición de sentencias SQL.

Operaciones con interacción del usuario

Inserción de un artículo introduciendo los datos por teclado.

Consulta de un artículo por su id desde Java.

Borrado de un registro por id.

Tabla actualizada tras las operaciones.

En la segunda parte del programa se trabajó con **interacción del usuario**, solicitando los datos por consola para insertar, consultar y borrar registros.

Para estas operaciones se utilizaron **PreparedStatement**, lo que permite trabajar con los datos de forma segura.

CAPTURAS DE PANTALLA AL FINAL

Llamada al procedimiento almacenado desde Java

Por último, el programa permite **insertar un artículo mediante el procedimiento almacenado**, solicitando los datos al usuario y realizando la llamada desde Java mediante un **CallableStatement**.

De esta forma se comprueba la correcta integración entre la base de datos y el programa Java, finalizando el ejercicio con la inserción correcta del registro y la visualización del resultado en la tabla.

CAPTURAS DE PANTALLA AL FINAL

“Código Java principal”

```
package ferreteria;

import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class laFerreteria {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/ferreteria";
        String user = "root";    // (o 'selma' si te funciona)
        String password = "";    // (o 'selma26*' si te funciona)

        // SQL preparadas
        String sqlInsertaP = "INSERT INTO articulos (idArticulo, descripcion, pvp, stock) VALUES (?,?,?,?)";
        String sqlSelectIdP = "SELECT * FROM articulos WHERE idArticulo = ?";
        String sqlBorraId = "DELETE FROM articulos WHERE idArticulo = ?";
        String sqlUpdateId = "UPDATE articulos SET descripcion = ?, pvp = ?, stock = ? WHERE idArticulo = ?";
        String sqlProcedimiento = "{call insertarArticulo(?, ?, ?, ?)}";

        try (Scanner teclado = new Scanner(System.in);
            Connection con = DriverManager.getConnection(url, user, password);
            Statement st = con.createStatement();
            PreparedStatement psInsert = con.prepareStatement(sqlInsertaP);
            PreparedStatement psSelectId = con.prepareStatement(sqlSelectIdP);
            PreparedStatement psDeleteId = con.prepareStatement(sqlBorraId);
            PreparedStatement psUpdateId = con.prepareStatement(sqlUpdateId);
            CallableStatement cstmt = con.prepareCall(sqlProcedimiento)) {

            // SIN interacción

            // Borra todos los registros
            st.executeUpdate("DELETE FROM articulos");
            System.out.println("Borrado hecho\n");

            // Inserta 3 registros con IGNORE
            String sqlInserta3 = ""
                INSERT IGNORE INTO articulos (idArticulo, descripcion, pvp, stock)
                VALUES (2, 'Llave Allen', 3.5, 50),
                    (3, 'Tornillos 5mm', 9.9, 200),
                    (4, 'Cinta aislante', 2.2, 80)
                """,
            st.executeUpdate(sqlInserta3);
```

```

System.out.println("Inserción de 3 registros hecha\n");

// Muestra todos los registros
MostrandoRegistros.mostrandoRegistros(con);

// Modifica un registro buscando por id (UPDATE por idArticulo)
// Ejemplo: modificar el id 3
psUpdateId.setString(1, "Tornillos 6mm");
psUpdateId.setDouble(2, 11.5);
psUpdateId.setInt(3, 180);
psUpdateId.setInt(4, 3);
int modificados = psUpdateId.executeUpdate();
System.out.println("Registros modificados: " + modificados + "\n");

// Muestra todos los registros
MostrandoRegistros.mostrandoRegistros(con);

// CON interacción

// Pide datos al usuario e inserta un registro
System.out.println("Inserta un artículo (id): ");
int id = Integer.parseInt(teclado.nextLine().trim());

System.out.println("Descripción: ");
String descripcion = teclado.nextLine().trim();

System.out.println("PVP (ej: 1900.4): ");
double pvp = Double.parseDouble(teclado.nextLine().trim());

System.out.println("Stock: ");
int stock = Integer.parseInt(teclado.nextLine().trim());

psInsert.setInt(1, id);
psInsert.setString(2, descripcion);
psInsert.setDouble(3, pvp);
psInsert.setInt(4, stock);
psInsert.executeUpdate();

// Muestra todos
MostrandoRegistros.mostrandoRegistros(con);

// Pide id y muestra por ID
System.out.println("Introduce un id para mostrar el registro:");
id = Integer.parseInt(teclado.nextLine().trim());

psSelectId.setInt(1, id);
try (ResultSet rs = psSelectId.executeQuery()) {
    boolean encontrado = false;
    while (rs.next()) {
        encontrado = true;
    }
}

```

```

        System.out.println("idArticulo: " + rs.getInt("idArticulo")
            + ", descripcion: " + rs.getString("descripcion")
            + ", pvp: " + rs.getDouble("pvp")
            + ", stock: " + rs.getInt("stock"));
    }
    if (!encontrado) System.out.println("No existe ese id.");
    System.out.println();
}

// Pide id y borra por ID
System.out.println("Introduce un id para borrar el registro:");
id = Integer.parseInt(teclado.nextLine().trim());

psDeleteId.setInt(1, id);
int borrados = psDeleteId.executeUpdate();
System.out.println("Registros borrados: " + borrados + "\n");
// Muestra todos
MostrandoRegistros.mostrandoRegistros(con);
// Llama al procedimiento almacenado con datos del usuario
System.out.println("Insertar con PROCEDIMIENTO (id): ");
id = Integer.parseInt(teclado.nextLine().trim());

System.out.println("Descripción: ");
descripcion = teclado.nextLine().trim();

System.out.println("PVP: ");
pvp = Double.parseDouble(teclado.nextLine().trim());

System.out.println("Stock: ");
stock = Integer.parseInt(teclado.nextLine().trim());

cstmt.setInt(1, id);
cstmt.setString(2, descripcion);
cstmt.setDouble(3, pvp);
cstmt.setInt(4, stock);
cstmt.execute();

// Muestra todos
MostrandoRegistros.mostrandoRegistros(con);

} catch (SQLException e) {
    System.out.println("Conexión no correcta / Error SQL");
    System.out.println(e.getMessage());
} catch (NumberFormatException e) {
    System.out.println("Has escrito un número con formato incorrecto.");
}
}
}

```

```

pom.xml [FerreteriaBDD JDBC] x laFerreteria.java x MostrandoRegistros.java x
Source History
1 package ferreteria;
2
3 import java.sql.CallableStatement;
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 import java.util.Scanner;
11
12 public class laFerreteria {
13
14     public static void main(String[] args) {
15
16         String url = "jdbc:mysql://localhost:3306/ferreteria";
17         String user = "root"; // (o 'selma' si te funciona)
18         String password = ""; // (o 'selma26*' si te funciona)
19
20         // SQL preparadas
21         String sqlInsertaP = "INSERT INTO articulos (idArticulo, descripcion, pvp, stock) VALUES (?, ?, ?, ?)";
22         String sqlSelectIdP = "SELECT * FROM articulos WHERE idArticulo = ?";
23         String sqlBorraId = "DELETE FROM articulos WHERE idArticulo = ?";
24         String sqlUpdateId = "UPDATE articulos SET descripcion = ?, pvp = ?, stock = ? WHERE idArticulo = ?";
25         String sqlProcedimiento = "{call insertarArticulo(?, ?, ?, ?)}";
26
27         try (Scanner teclado = new Scanner(System.in);
28             Connection con = DriverManager.getConnection(url, user, password);
29             Statement st = con.createStatement();
30             PreparedStatement psInsert = con.prepareStatement(sqlInsertaP);
31             PreparedStatement psSelectId = con.prepareStatement(sqlSelectIdP);
32             PreparedStatement psDeleteId = con.prepareStatement(sqlBorraId);
33             PreparedStatement psUpdateId = con.prepareStatement(sqlUpdateId);
34             CallableStatement cstmt = con.prepareCall(sqlProcedimiento)) {
35
36             // -----
37             // SIN interacción
38             // -----
39
40             // Borra todos los registros
41             st.executeUpdate("DELETE FROM articulos");
42             System.out.println("Borrado hecho\n");
43
44             // Inserta 3 registros con IGNORE
45             String sqlInserta3 = ""
46                 + "INSERT IGNORE INTO articulos (idArticulo, descripcion, pvp, stock) "
47                 + "VALUES (2, 'Llave Allen', 3.5, 50), "
48                 + "        (3, 'Tornillos 5mm', 9.9, 200), "
49                 + "        (4, 'Cinta aislante', 2.2, 80) "
50                 + ""
51             ;
52             st.executeUpdate(sqlInserta3);
53             System.out.println("Inserción de 3 registros hecha\n");
54
55             // Muestra todos los registros
56             MostrandoRegistros.mostrandoRegistros(con);
57
58             // Modifica un registro buscando por id (UPDATE por idArticulo)
59             // Ejemplo: modificar el id 3
60             psUpdateId.setString(1, "Tornillos 6mm");
61             psUpdateId.setDouble(2, 11.5);
62             psUpdateId.setInt(3, 180);
63             psUpdateId.setInt(4, 3);
64             int modificados = psUpdateId.executeUpdate();
65             System.out.println("Registros modificados: " + modificados + "\n");
66
67             // Muestra todos los registros
68             MostrandoRegistros.mostrandoRegistros(con);
69
70             // -----
71             // CON interacción
72             // -----
73
74             // Pide datos al usuario e inserta un registro
75             System.out.println("Inserta un articulo (id): ");
76             int id = Integer.parseInt(teclado.nextLine().trim());
77
78             System.out.println("Descripción: ");
79             String descripcion = teclado.nextLine().trim();
80
81             System.out.println("PVP (ej: 1900.4): ");
82             double pvp = Double.parseDouble(teclado.nextLine().trim());
83
84             System.out.println("Stock: ");
85             int stock = Integer.parseInt(teclado.nextLine().trim());
86
87             psInsert.setInt(1, id);
88             psInsert.setString(2, descripcion);
89             psInsert.setDouble(3, pvp);
90             psInsert.setInt(4, stock);
91             psInsert.executeUpdate();
92
93             // Muestra todos
94             MostrandoRegistros.mostrandoRegistros(con);
95
96             // Pide id y muestra por ID
97             System.out.println("Introduce un id para mostrar el registro:");
98             id = Integer.parseInt(teclado.nextLine().trim());

```



```

99     psSelectId.setInt(1, id);
100     try (ResultSet rs = psSelectId.executeQuery()) {
101         boolean encontrado = false;
102         while (rs.next()) {
103             encontrado = true;
104             System.out.println("idArticulo: " + rs.getInt("idArticulo")
105                 + ", descripcion: " + rs.getString("descripcion")
106                 + ", pvp: " + rs.getDouble("pvp")
107                 + ", stock: " + rs.getInt("stock"));
108         }
109         if (!encontrado) System.out.println("No existe ese id.");
110         System.out.println();
111     }
112
113     // Pide id y borra por ID
114     System.out.println("Introduce un id para borrar el registro:");
115     id = Integer.parseInt(teclado.nextLine().trim());
116
117     psDeleteId.setInt(1, id);
118     int borrados = psDeleteId.executeUpdate();
119     System.out.println("Registros borrados: " + borrados + "\n");
120
121     // Muestra todos
122     MostrandoRegistros.mostrandoRegistros(con);
123
124     // Llama al procedimiento almacenado con datos del usuario
125     System.out.println("Insertar con PROCEDIMIENTO (id): ");
126     id = Integer.parseInt(teclado.nextLine().trim());
127
128     System.out.println("Descripción: ");
129     descripcion = teclado.nextLine().trim();
130
131     System.out.println("PVP: ");
132     pvp = Double.parseDouble(teclado.nextLine().trim());
133
134     System.out.println("Stock: ");
135     stock = Integer.parseInt(teclado.nextLine().trim());
136
137     pstmt.setInt(1, id);
138     pstmt.setString(2, descripcion);
139     pstmt.setDouble(3, pvp);
140     pstmt.setInt(4, stock);
141     pstmt.executeUpdate();
142
143     // Muestra todos
144     MostrandoRegistros.mostrandoRegistros(con);
145
146 } catch (SQLException e) {
147     System.out.println("Conexión no correcta / Error SQL");
148     System.out.println(e.getMessage());
149 } catch (NumberFormatException e) {
150     System.out.println("Has escrito un número con formato incorrecto.");
151 }
152 }
153 }
154

```

Output - Run (laFerreteria) x

“Clase MostrandoRegistros”

```
package ferreteria;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class MostrandoRegistros {

    public static void mostrandoRegistros(Connection conexion) {
        String sqlSelect = "SELECT * FROM articulos";

        try (Statement stmt = conexion.createStatement();
            ResultSet rs = stmt.executeQuery(sqlSelect)) {

            System.out.println("TABLA ARTICULOS (mediante método)");
            System.out.println("-----");

            while (rs.next()) {
                System.out.println("idArticulo: " + rs.getInt("idArticulo")
                    + ", descripcion: " + rs.getString("descripcion")
                    + ", pvp: " + rs.getDouble("pvp")
                    + ", stock: " + rs.getInt("stock"));
            }
            System.out.println();

        } catch (SQLException e) {
            System.out.println("Error al mostrar registros");
            System.out.println(e.getMessage());
        }
    }
}
```

```

pom.xml [FerreteriaBDD/JDBC] x laFerreteria.java x MostrandoRegistros.java x
Source History
1 package ferreteria;
2
3 import java.sql.Connection;
4 import java.sql.ResultSet;
5 import java.sql.SQLException;
6 import java.sql.Statement;
7
8 public class MostrandoRegistros {
9
10     public static void mostrandoRegistros(Connection conexion) {
11         String sqlSelect = "SELECT * FROM articulos";
12
13         try (Statement stmt = conexion.createStatement();
14             ResultSet rs = stmt.executeQuery(sqlSelect)) {
15
16             System.out.println("TABLA ARTICULOS (mediante método)");
17             System.out.println("-----");
18
19             while (rs.next()) {
20                 System.out.println("idArtículo: " + rs.getInt("idArtículo")
21                                     + ", descripción: " + rs.getString("descripcion")
22                                     + ", pvp: " + rs.getDouble("pvp")
23                                     + ", stock: " + rs.getInt("stock"));
24             }
25             System.out.println();
26
27         } catch (SQLException e) {
28             System.out.println("Error al mostrar registros");
29             System.out.println(e.getMessage());
30         }
31     }
32 }
33
34
Output - Run (laFerreteria) x

```

CAPTURAS EJECUCIÓN

Inserción de un artículo introduciendo los datos por teclado.

Consulta de un artículo por su id desde Java.

```

pom.xml [FerreteriaBDD/JDBC] x laFerreteria.java x MostrandoRegistros.java x
Source History
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 import java.util.Scanner;
11
12 public class laFerreteria {
13
14     public static void main(String[] args) {
15
16         String url = "jdbc:mysql://localhost:3306/ferreteria";
17         String user = "root"; // (o 'selma' si te funciona)
18         String password = ""; // (o 'selma26*' si te funciona)
19
20         // SQL preparadas
21         String sqlInsertaP = "INSERT INTO articulos (idArtículo, descripcion, pvp, stock) VALUES (?, ?, ?, ?)";
22         String sqlSelectIdP = "SELECT * FROM articulos WHERE idArtículo = ?";
23         String sqlBorraId = "DELETE FROM articulos WHERE idArtículo = ?";
24         String sqlUpdateId = "UPDATE articulos SET descripcion = ?, pvp = ?, stock = ? WHERE idArtículo = ?";
25         String sqlProcedimiento = "(call insertarArtículo(?, ?, ?, ?))";
26
27     }
28 }
29
Output - Run (laFerreteria) x

```

TABLA ARTICULOS (mediante método)

idArtículo: 2, descripción: Llave Allen, pvp: 3.5, stock: 50
idArtículo: 3, descripción: Tornillos 6mm, pvp: 11.5, stock: 180
idArtículo: 4, descripción: Cinta aislante, pvp: 2.2, stock: 80

Inserta un artículo (id):
10
Descripción:
Alicates universales
PVP (ej: 1900.4):
12.5
Stock:
8

```

Output - Run (laFerreteria) x

```

Stock:
8

TABLA ARTICULOS (mediante método)

idArtículo: 2, descripción: Llave Allen, pvp: 3.5, stock: 50
idArtículo: 3, descripción: Tornillos 6mm, pvp: 11.5, stock: 180
idArtículo: 4, descripción: Cinta aislante, pvp: 2.2, stock: 80
idArtículo: 10, descripción: Alicates universales, pvp: 12.5, stock: 8

Introduce un id para mostrar el registro:
10
idArtículo: 10, descripción: Alicates universales, pvp: 12.5, stock: 8

Introduce un id para borrar el registro:

Servidor: 127.0.0.1 » Base de datos: ferreteria » Tabla: articulos

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Disparadores

Mostrar ventana de consultas SQL

Mostrando filas 0 - 3 (total de 4, La consulta tardó 0,0011 segundos.)

SELECT * FROM articulos;

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	idArticulo	descripcion	pvp	stock
☐ Editar ☐ Copiar ☐ Borrar	2	Llave Allen	3.5	50
☐ Editar ☐ Copiar ☐ Borrar	3	Tornillos 6mm	11.5	180
☐ Editar ☐ Copiar ☐ Borrar	4	Cinta aislante	2.2	80
☐ Editar ☐ Copiar ☐ Borrar	10	Alicates universales	12.5	8

Seleccionar todo | Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Borrado de un registro por id.

Source History pom.xml [ferreteria88DDJDBC] x laferreteria.java x MostrandoRegistros.java x

```

7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 import java.util.Scanner;
11
12 public class laFerreteria {
13
14     public static void main(String[] args) {
15
16         String url = "jdbc:mysql://localhost:3306/ferreteria";
17         String user = "root"; // (o 'selma' si te funciona)
18         String password = ""; // (o 'selma26*' si te funciona)
19
20         // SQL preparadas
21         String sqlInsertaP = "INSERT INTO articulos (idArticulo, descripcion, pvp, stock) VALUES (?, ?, ?, ?)";
22         String sqlSelectIdP = "SELECT * FROM articulos WHERE idArticulo = ?";
23         String sqlBorraIdP = "DELETE FROM articulos WHERE idArticulo = ?";
24         String sqlUpdateIdP = "UPDATE articulos SET descripcion = ?, pvp = ?, stock = ? WHERE idArticulo = ?";
25         String sqlProcedimiento = "(call insertarArticulo(?, ?, ?, ?))";
26
27     }
28 }

```

Output - Run (laFerreteria) x

```

10 idArticulo: 10, descripcion: Alicates universales, pvp: 12.5, stock: 8
Introduce un id para borrar el registro:
10
Registros borrados: 1
TABLA ARTICULOS (mediante m#todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80
Insertar con PROCEDIMIENTO (id):

```

Servidor: 127.0.0.1 » Base de datos: ferreteria » Tabla: articulos

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Disparadores

Mostrar ventana de consultas SQL

Mostrando filas 0 - 2 (total de 3, La consulta tardó 0,0007 segundos.)

select * from articulos;

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	idArticulo	descripcion	pvp	stock
☐ Editar ☐ Copiar ☐ Borrar	2	Llave Allen	3.5	50
☐ Editar ☐ Copiar ☐ Borrar	3	Tornillos 6mm	11.5	180
☐ Editar ☐ Copiar ☐ Borrar	4	Cinta aislante	2.2	80

Seleccionar todo | Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

Llamada al procedimiento almacenado desde Java

```
pom.xml [FerreteriaABDD JDBC] x laFerreteria.java x MostrandoRegistros.java x
Source History
Output - Run (laFerreteria) x
Inserción de 3 registros hecha
-----
TABLA ARTICULOS (mediante m+todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 5.5, stock: 200
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80

Registros modificados: 1
-----
TABLA ARTICULOS (mediante m+todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80

Inserta un artículo (id):
20
Descripción:
Martillo percutor
PVP (ej: 1500.4):
1900.4
Stock:
2
TABLA ARTICULOS (mediante m+todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80
idArticulo: 20, descripcion: Martillo percutor, pvp: 1900.4, stock: 2

Introduce un id para mostrar el registro:
20
idArticulo: 20, descripcion: Martillo percutor, pvp: 1900.4, stock: 2
```

```
pom.xml [FerreteriaABDD JDBC] x laFerreteria.java x MostrandoRegistros.java x
Source History
Output - Run (laFerreteria) x
Introduce un id para mostrar el registro:
20
idArticulo: 20, descripcion: Martillo percutor, pvp: 1900.4, stock: 2

Introduce un id para borrar el registro:
20
Registros borrados: 1
-----
TABLA ARTICULOS (mediante m+todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80

Insertar con PROCEDIMIENTO (id):
20
Descripción:
Martillo percutor
PVP:
1900.4
Stock:
2
TABLA ARTICULOS (mediante m+todo)
-----
idArticulo: 2, descripcion: Llave Allen, pvp: 3.5, stock: 50
idArticulo: 3, descripcion: Tornillos 6mm, pvp: 11.5, stock: 180
idArticulo: 4, descripcion: Cinta aislante, pvp: 2.2, stock: 80
idArticulo: 20, descripcion: Martillo percutor, pvp: 1900.4, stock: 2

-----
BUILD SUCCESS
-----
Total time: 03:38 min
Finished at: 2026-02-04T11:37:44+01:00
-----
(1) 1026 INS
```

Servidor_127.0.0.1 > Base de datos: ferreteria > Tabla: articulos

ExaminarEstructuraSQLBuscarInsertarExportarImportarPrivilegiosOperacionesDisparadores

Mostrar ventana de consultas SQL

Mostrando filas 0 - 3 (total de 4, La consulta tardó 0,0010 segundos.)

`select * from articulos;`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo

 |

Número de filas: 25

 |

Filtrar filas: Buscar en esta tabla

 |

Ordenar según la clave: Ninguna

Opciones extra

idArticulo

descripcion

pvp

stock

☐

Editar

Copiar

Borrar

2 Llave Allen

3.5

50

☐

Editar

Copiar

Borrar

3 Tornillos 6mm

11.5

180

☐

Editar

Copiar

Borrar

4 Cinta aislante

2.2

80

☐

Editar

Copiar

Borrar

20 Martillo percutor

1900.4

2

Seleccionar todo

 |

Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo

 |

Número de filas: 25

 |

Filtrar filas: Buscar en esta tabla

 |

Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

Consola